JAVA FULL STACK + DSA MASTERY ROADMAP

Beginner → 10+ LPA | Version 2.0 (Expanded)

Prepared for: **Nishanth P**

EIE, Sri Ramakrishna Engineering College (SREC), Coimbatore

LinkedIn: linkedin.com/in/nishanth-p-dev | GitHub: github.com/nishanth-1431

11	451+	6-8	10+ LPA
Phases	Problems / Tasks	Months	Target

This is version 2 of your roadmap — expanded from the original 7-phase / 150-problem plan into a full **Java Full Stack interview toolkit**: Core DSA patterns (Phases 1-8), SQL query patterns (Phase 9), Spring Boot & backend design patterns (Phase 10), and System Design basics (Phase 11). Built for campus placement targeting MNC companies (TCS, Infosys, Wipro) as tier-1, Zoho/Freshworks as tier-2, and Walmart Global Tech / SAP Labs / Razorpay as dream-tier. Low CGPA is NOT a blocker — strong DSA + backend fundamentals + GitHub + projects is the play.

YOUR NON-NEGOTIABLE RULES

RULE 1	**Never copy-paste solutions** Read the problem. Think for at least 20 minutes. Look at hints only — never full solutions. The struggle is where learning happens.
RULE 2	**GitHub every single day** Even solving 1 problem = 1 commit. Low CGPA gets forgiven when recruiters see 180+ days of green squares.
RULE 3	**Pattern first, brute force never** Before writing any code, ask: which pattern does this problem look like? Training your brain to pattern-match is the actual skill that gets you hired.
RULE 4	**Don't just memorize DSA — ship backend proof** For every 2 weeks of DSA, build or extend one small Spring Boot feature. Interviewers at Zoho/Freshworks/dream-tier test both.

TABLE OF CONTENTS

PHASE 1 Java Syntax + Array/String Foundations

Week 1-2

Master Java's syntax building blocks used in every DSA problem

Before LeetCode, get comfortable with: int/long/double, String methods, arrays (1D/2D), ArrayList, HashMap basics, for/while loops, methods, Math.max/min/abs, Integer.MAX_VALUE, System.out.println, Scanner, and Big-O notation basics (O(n), O(log n), O(n^2)).

Arrays & Strings Warm-up

Tip: Solve every Easy here before Phase 2. Target: 2/day.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Two Sum	1	Easy	HashMap basics
2	Contains Duplicate	217	Easy	HashSet
3	Valid Anagram	242	Easy	Char frequency array
4	Valid Palindrome	125	Easy	Two pointer preview
5	Product of Array Except Self	238	Medium	Prefix/Suffix product
6	Merge Intervals	56	Medium	Sorting + greedy
7	Rotate Image	48	Medium	In-place matrix rotation
8	Spiral Matrix	54	Medium	Boundary simulation
9	Fizz Buzz	412	Easy	Basic loop logic
10	Fibonacci Number	509	Easy	Recursion intro
11	Single Number	136	Easy	XOR bit trick
12	Missing Number	268	Easy	Gauss formula / XOR
13	Move Zeroes	283	Easy	Two pointer in-place
14	Plus One	66	Easy	Array carry logic
15	Best Time to Buy and Sell Stock	121	Easy	Single pass min tracking
16	Majority Element	169	Easy	Boyer-Moore voting
17	Rotate Array	189	Medium	Cyclic rotation / reversal
18	Set Matrix Zeroes	73	Medium	In-place marking
19	Maximum Subarray	53	Medium	Kadane's algorithm
20	Pascal's Triangle	118	Easy	2D array build

PHASE 2 Two Pointers + Sliding Window + Prefix Sum + Binary Search

Week 3-5

4 of the most common patterns in MNC screening rounds

Two Pointers

Tip: Use when array is sorted, or you need a pair/triplet with a condition, or in-place reversal.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Two Sum II - Input Array Is Sorted	167	Easy	Classic two pointer
2	3Sum	15	Medium	Sort + two pointer
3	3Sum Closest	16	Medium	Two pointer variant
4	4Sum	18	Medium	Two pointer generalized
5	Container With Most Water	11	Medium	Max area two pointer
6	Trapping Rain Water	42	Hard	Two pointer / stack
7	Sort Colors	75	Medium	Dutch flag 3-pointer
8	Merge Sorted Array	88	Easy	Two pointer merge
9	Squares of a Sorted Array	977	Easy	Two pointer from ends
10	Remove Duplicates from Sorted Array	26	Easy	Slow-fast pointer
11	Remove Duplicates from Sorted Array II	80	Medium	Slow-fast pointer + count
12	Reverse String	344	Easy	Two pointer swap
13	Valid Palindrome II	680	Easy	Two pointer + one skip
14	Backspace String Compare	844	Easy	Two pointer from back
15	Boats to Save People	881	Medium	Greedy two pointer
16	Partition Labels	763	Medium	Two pointer + last-seen map
17	Sort Array By Parity	905	Easy	Two pointer partition
18	Interval List Intersections	986	Medium	Two pointer on intervals

Sliding Window

Tip: Fixed window = fixed size k. Variable window = expand right, shrink left when condition breaks.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Maximum Average Subarray I	643	Easy	Fixed window
2	Longest Substring Without Repeating Characters	3	Medium	Variable window + set
3	Longest Repeating Character Replacement	424	Medium	Variable window + freq
4	Permutation in String	567	Medium	Fixed window + anagram
5	Minimum Window Substring	76	Hard	Variable window + map
6	Sliding Window Maximum	239	Hard	Deque / monotonic queue
7	Find All Anagrams in a String	438	Medium	Fixed window + freq

#	Problem Name	LC No.	Difficulty	Key Concept
8	Max Consecutive Ones III	1004	Medium	Variable window at most k
9	Fruit Into Baskets	904	Medium	Variable window, 2 types
10	Longest Subarray of 1's After Deleting One Element	1493	Medium	Window with 1 deletion
11	Subarrays with K Different Integers	992	Hard	At-most(k) - At-most(k-1) trick
12	Longest Substring with At Most Two Distinct Characters	159	Medium	Variable window + freq map
13	Binary Subarrays With Sum	930	Medium	At-most window trick
14	Grumpy Bookstore Owner	1052	Medium	Fixed window gain
15	Longest Continuous Increasing Subsequence	674	Easy	Simple expanding window
16	Count Number of Nice Subarrays	1248	Medium	At-most(k) window trick
17	Maximum Points You Can Obtain from Cards	1423	Medium	Window from both ends

Prefix Sum

Tip: Build prefix[] where prefix[i] = sum of first i elements. Query = prefix[r] - prefix[l-1].

#	Problem Name	LC No.	Difficulty	Key Concept
1	Range Sum Query - Immutable	303	Easy	Classic prefix sum
2	Range Sum Query 2D - Immutable	304	Medium	2D prefix sum
3	Subarray Sum Equals K	560	Medium	Prefix sum + hashmap
4	Subarray Sums Divisible by K	974	Medium	Prefix sum + mod
5	Continuous Subarray Sum	523	Medium	Prefix sum + mod
6	Contiguous Array	525	Medium	Prefix sum trick (0/-1)
7	Running Sum of 1d Array	1480	Easy	Basic prefix
8	Find Pivot Index	724	Easy	Left/right prefix compare
9	Maximum Size Subarray Sum Equals k	325	Medium	Prefix sum + first-seen map
10	Minimum Value to Get Positive Step by Step Sum	1413	Easy	Running prefix minimum
11	Corporate Flight Bookings	1109	Medium	Difference array
12	Number of Subarrays with Bounded Maximum	795	Medium	Prefix window counting

Binary Search

Tip: Template: lo=0, hi=n-1, mid=(lo+hi)/2. Adjust lo/hi based on condition. Master 'BS on answer space'.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Binary Search	704	Easy	Classic template
2	Search Insert Position	35	Easy	Lower bound BS
3	Sqrt(x)	69	Easy	BS on answer space
4	Find First and Last Position of Element	34	Medium	BS boundaries
5	Search a 2D Matrix	74	Medium	BS on flattened matrix
6	Search a 2D Matrix II	240	Medium	Staircase search

#	Problem Name	LC No.	Difficulty	Key Concept
7	Find Minimum in Rotated Sorted Array	153	Medium	BS with rotation
8	Find Minimum in Rotated Sorted Array II	154	Hard	BS with duplicates
9	Search in Rotated Sorted Array	33	Medium	BS with rotation
10	Search in Rotated Sorted Array II	81	Medium	BS with duplicates
11	Koko Eating Bananas	875	Medium	BS on answer space
12	Capacity to Ship Packages in D Days	1011	Medium	BS on answer space
13	Minimum Number of Days to Make m Bouquets	1482	Medium	BS on answer space
14	Find Peak Element	162	Medium	BS on unsorted
15	Kth Smallest Element in a Sorted Matrix	378	Medium	BS on value range
16	H-Index II	275	Medium	BS on sorted citations
17	Split Array Largest Sum	410	Hard	BS on answer space
18	Median of Two Sorted Arrays	4	Hard	BS advanced partition

PHASE 3 Hashing + Stack + Queue + Monotonic Stack

Week 6-8

Most frequently tested patterns in TCS/Infosys/Wipro/Zoho screening

HashMap / HashSet

Tip: getOrDefault() and computeIfAbsent() are your friends for O(1) counting and lookups.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Two Sum	1	Easy	HashMap index store
2	Group Anagrams	49	Medium	Sort key + grouping
3	Longest Consecutive Sequence	128	Medium	HashSet O(n) solution
4	Top K Frequent Elements	347	Medium	Freq map + bucket sort
5	Word Pattern	290	Easy	Bijection mapping
6	Isomorphic Strings	205	Easy	Two-way char map
7	Ransom Note	383	Easy	Freq counting
8	4Sum II	454	Medium	HashMap pair sum
9	Insert Delete GetRandom O(1)	380	Medium	HashMap + ArrayList
10	Subarray Sum Equals K	560	Medium	Prefix + hashmap
11	Longest Substring Without Repeating Characters	3	Medium	HashMap last-index
12	First Unique Character in a String	387	Easy	Freq map
13	Sort Characters By Frequency	451	Medium	Freq map + sort
14	Design HashMap	706	Easy	Bucket design
15	Contains Duplicate II	219	Easy	HashMap index window
16	Brick Wall	554	Medium	Gap-count hashmap
17	Fraction to Recurring Decimal	166	Medium	Remainder hashmap
18	Continuous Subarray Sum	523	Medium	Prefix + mod hashmap

Stack

Tip: Use Deque stack = new ArrayDeque<>() — never java.util.Stack (synchronized, slow).

#	Problem Name	LC No.	Difficulty	Key Concept
1	Valid Parentheses	20	Easy	Classic stack match
2	Min Stack	155	Medium	Stack with min tracking
3	Evaluate Reverse Polish Notation	150	Medium	Stack expression eval
4	Daily Temperatures	739	Medium	Monotonic stack
5	Next Greater Element I	496	Easy	Monotonic stack + map
6	Next Greater Element II	503	Medium	Circular monotonic stack
7	Largest Rectangle in Histogram	84	Hard	Monotonic stack classic
8	Maximal Rectangle	85	Hard	Stack on 2D histogram

#	Problem Name	LC No.	Difficulty	Key Concept
9	Implement Queue using Stacks	232	Easy	Two-stack queue
10	Decode String	394	Medium	Stack string building
11	Simplify Path	71	Medium	Stack path parsing
12	Asteroid Collision	735	Medium	Stack simulation
13	Remove All Adjacent Duplicates In String	1047	Easy	Stack char compare
14	Basic Calculator	224	Hard	Stack + sign tracking
15	Basic Calculator II	227	Medium	Stack + operator precedence
16	Online Stock Span	901	Medium	Monotonic stack design
17	Remove K Digits	402	Medium	Monotonic stack greedy
18	Trapping Rain Water	42	Hard	Monotonic stack variant

Queue / Deque / BFS Preview

Tip: ArrayDeque doubles as both stack and queue — know when to use pollFirst vs pollLast.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Implement Stack using Queues	225	Easy	Two-queue stack
2	Number of Recent Calls	933	Easy	Queue sliding window
3	Moving Average from Data Stream	346	Easy	Queue fixed size
4	Sliding Window Maximum	239	Hard	Deque monotonic
5	Design Circular Deque	641	Medium	Deque design
6	Design Circular Queue	622	Medium	Array-based queue
7	Task Scheduler	621	Medium	Greedy + queue/heap
8	Dota2 Senate	649	Medium	Queue simulation
9	Design Front Middle Back Queue	1670	Medium	Two-deque design
10	First Unique Number	1429	Medium	Queue + hashmap design

PHASE 4 Recursion + Backtracking + Linked List

Week 9-11

Build the mental model for recursive thinking and pointer manipulation

Recursion Basics

Tip: Every recursive function needs: a base case, and a call that moves toward it.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Fibonacci Number	509	Easy	Base case + recursive call
2	Reverse String	344	Easy	Recursion on array
3	Reverse Linked List	206	Easy	Recursion on list
4	Maximum Depth of Binary Tree	104	Easy	Tree recursion
5	Merge Two Sorted Lists	21	Easy	Recursive merge
6	Climbing Stairs	70	Easy	Recursion to DP bridge
7	Power of Three	326	Easy	Recursive divisibility
8	Pow(x, n)	50	Medium	Fast exponentiation recursion
9	Sqrt(x)	69	Easy	Recursive/BS combo
10	Tower of Hanoi (GfG classic)	N/A	Medium	Classic recursion drill

Backtracking

Tip: Template: choose -> explore -> un-choose. Always draw the decision tree before coding.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Subsets	78	Medium	Backtrack include/exclude
2	Subsets II	90	Medium	Backtrack + skip dup
3	Permutations	46	Medium	Backtrack + visited[]
4	Permutations II	47	Medium	Backtrack + sort + skip
5	Combination Sum	39	Medium	Backtrack reuse allowed
6	Combination Sum II	40	Medium	Backtrack no reuse
7	Combinations	77	Medium	Backtrack fixed size
8	Word Search	79	Medium	DFS + backtrack on grid
9	Word Search II	212	Hard	Trie + backtrack on grid
10	N-Queens	51	Hard	Classic backtrack puzzle
11	N-Queens II	52	Hard	Backtrack counting
12	Letter Combinations of a Phone Number	17	Medium	Backtrack string build
13	Palindrome Partitioning	131	Medium	Backtrack + isPalin check
14	Generate Parentheses	22	Medium	Backtrack + balance count
15	Sudoku Solver	37	Hard	Backtrack + constraint check
16	Restore IP Addresses	93	Medium	Backtrack + segment check
17	Beautiful Arrangement	526	Medium	Backtrack + pruning

#	Problem Name	LC No.	Difficulty	Key Concept
18	Matchsticks to Square	473	Medium	Backtrack + pruning

Linked List + Fast-Slow Pointer

Tip: Always draw pointer diagrams on paper before coding linked list problems.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Reverse Linked List	206	Easy	Iterative + recursive
2	Reverse Linked List II	92	Medium	Reverse sub-section
3	Merge Two Sorted Lists	21	Easy	Two pointer merge
4	Merge K Sorted Lists	23	Hard	PriorityQueue / divide
5	Linked List Cycle	141	Easy	Fast-slow pointer
6	Linked List Cycle II	142	Medium	Fast-slow cycle entry
7	Remove Nth Node From End	19	Medium	Two pointer n-gap
8	Middle of the Linked List	876	Easy	Fast-slow mid
9	Reorder List	143	Medium	Find mid + reverse + merge
10	Add Two Numbers	2	Medium	Carry simulation
11	Add Two Numbers II	445	Medium	Stack + carry simulation
12	Intersection of Two Linked Lists	160	Easy	Two pointer length trick
13	Copy List with Random Pointer	138	Medium	HashMap node mapping
14	Palindrome Linked List	234	Easy	Fast-slow + reverse half
15	Odd Even Linked List	328	Medium	Two pointer partition
16	Swap Nodes in Pairs	24	Medium	Pointer manipulation
17	Rotate List	61	Medium	Cycle + break
18	Flatten a Multilevel Doubly Linked List	430	Medium	DFS on list

PHASE 5 Trees + BST + Trie + BFS/DFS

Week 12-15

Trees are in 30%+ of all placement interview questions

Binary Tree Traversals

Tip: Master recursive AND iterative (stack-based) versions of all three traversals.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Binary Tree Inorder Traversal	94	Easy	Recursive + iterative
2	Binary Tree Preorder Traversal	144	Easy	Recursive + iterative
3	Binary Tree Postorder Traversal	145	Easy	Recursive + two-stack
4	Binary Tree Level Order Traversal	102	Medium	BFS with queue
5	Binary Tree Zigzag Level Order	103	Medium	BFS + direction toggle
6	Binary Tree Level Order Traversal II	107	Medium	BFS reverse result
7	Binary Tree Right Side View	199	Medium	BFS last per level
8	Maximum Depth of Binary Tree	104	Easy	DFS/BFS depth
9	Minimum Depth of Binary Tree	111	Easy	BFS shortest path
10	Diameter of Binary Tree	543	Medium	DFS height + max
11	Average of Levels in Binary Tree	637	Easy	BFS level average
12	N-ary Tree Level Order Traversal	429	Medium	BFS on n-ary tree
13	Vertical Order Traversal of a Binary Tree	987	Hard	BFS/DFS + column map
14	Boundary of Binary Tree	545	Medium	DFS boundary traversal

Binary Tree Properties

Tip: Most of these reduce to a DFS that returns extra info up the call stack.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Same Tree	100	Easy	Recursive equality check
2	Symmetric Tree	101	Easy	Mirror recursion
3	Balanced Binary Tree	110	Easy	Height check recursion
4	Path Sum	112	Easy	DFS target subtraction
5	Path Sum II	113	Medium	DFS + backtrack paths
6	Path Sum III	437	Medium	Prefix sum on tree
7	Binary Tree Maximum Path Sum	124	Hard	DFS gain calculation
8	Lowest Common Ancestor of a Binary Tree	236	Medium	DFS LCA classic
9	Serialize and Deserialize Binary Tree	297	Hard	BFS/DFS encode decode
10	Invert Binary Tree	226	Easy	DFS swap children
11	Construct Binary Tree from Preorder and Inorder	105	Medium	Recursive build
12	Construct Binary Tree from Inorder and Postorder	106	Medium	Recursive build

#	Problem Name	LC No.	Difficulty	Key Concept
13	Flatten Binary Tree to Linked List	114	Medium	DFS in-place reshape
14	Sum Root to Leaf Numbers	129	Medium	DFS path accumulation
15	Count Good Nodes in Binary Tree	1448	Medium	DFS with running max

Binary Search Tree (BST)

Tip: BST inorder traversal gives sorted output — memorise this single fact, it solves 5+ problems.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Search in a BST	700	Easy	BST property: go left/right
2	Insert into a BST	701	Medium	BST recursive insert
3	Delete Node in a BST	450	Medium	Find inorder successor
4	Validate Binary Search Tree	98	Medium	BST min/max range check
5	Kth Smallest Element in BST	230	Medium	Inorder = sorted
6	Lowest Common Ancestor of a BST	235	Medium	BST LCA shortcut
7	Convert Sorted Array to BST	108	Easy	Mid as root recursion
8	Convert BST to Sorted Doubly Linked List	426	Medium	Inorder + link building
9	Trim a Binary Search Tree	669	Medium	BST bounded recursion
10	Two Sum IV - Input is a BST	653	Easy	Inorder + two pointer / set
11	Recover Binary Search Tree	99	Hard	Inorder + swap detection
12	Balance a Binary Search Tree	1382	Medium	Inorder + rebuild

Trie (Prefix Tree)

Tip: Backbone of autocomplete, spellcheck, and IP routing problems.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Implement Trie (Prefix Tree)	208	Medium	Classic trie design
2	Design Add and Search Words Data Structure	211	Medium	Trie + wildcard DFS
3	Word Search II	212	Hard	Trie + grid backtrack
4	Replace Words	648	Medium	Trie prefix lookup
5	Longest Word in Dictionary	720	Medium	Trie + BFS/DFS
6	Maximum XOR of Two Numbers in an Array	421	Medium	Binary trie
7	Search Suggestions System	1268	Medium	Trie / sorted prefix search
8	Map Sum Pairs	677	Medium	Trie with prefix sum

PHASE 6 Heaps + Greedy + Dynamic Programming

Week 16-20

DP separates MNC-ready candidates from the rest

DP mindset: Every DP problem = recursion + memoization OR tabulation. Start with brute force recursion, add a memo table, then convert to bottom-up. Never jump straight to the DP table.

Heap / Priority Queue

Tip: Java: PriorityQueue<> defaults to min-heap. Use Collections.reverseOrder() or a custom comparator for max-heap.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Kth Largest Element in an Array	215	Medium	Min-heap of size k
2	Top K Frequent Elements	347	Medium	Heap + freq map
3	Merge K Sorted Lists	23	Hard	Min-heap merge
4	Find Median from Data Stream	295	Hard	Two heaps
5	Task Scheduler	621	Medium	Max-heap greedy
6	K Closest Points to Origin	973	Medium	Max-heap of size k
7	Last Stone Weight	1046	Easy	Max-heap simulation
8	Kth Smallest Element in a Sorted Matrix	378	Medium	Heap on matrix
9	Reorganize String	767	Medium	Max-heap greedy placement
10	Ugly Number II	264	Medium	Heap / 3-pointer DP
11	Sliding Window Median	480	Hard	Two heaps + lazy delete
12	Meeting Rooms II	253	Medium	Min-heap of end times
13	The Skyline Problem	218	Hard	Heap + sweep line
14	IPO	502	Hard	Two heaps greedy

Greedy

Tip: Ask: 'If I make the locally best choice now, can I prove it never hurts the global answer?'

#	Problem Name	LC No.	Difficulty	Key Concept
1	Jump Game	55	Medium	Greedy reachability
2	Jump Game II	45	Medium	Greedy min jumps
3	Gas Station	134	Medium	Greedy total/current check
4	Candy	135	Hard	Two-pass greedy
5	Merge Intervals	56	Medium	Sort + greedy merge
6	Non-overlapping Intervals	435	Medium	Sort by end + greedy
7	Minimum Number of Arrows to Burst Balloons	452	Medium	Sort + greedy
8	Partition Labels	763	Medium	Greedy last-index
9	Boats to Save People	881	Medium	Two pointer greedy
10	Lemonade Change	860	Easy	Greedy simulation
11	Assign Cookies	455	Easy	Sort + two pointer greedy

#	Problem Name	LC No.	Difficulty	Key Concept
12	Queue Reconstruction by Height	406	Medium	Sort + greedy insert
13	Task Scheduler	621	Medium	Greedy + heap
14	Minimum Platforms (GfG classic)	N/A	Medium	Sort + two-pointer greedy

Dynamic Programming - 1D

Tip: dp[i] should depend only on a few previous states. Write the recurrence in words first.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Climbing Stairs	70	Easy	dp[i]=dp[i-1]+dp[i-2]
2	House Robber	198	Medium	dp[i]=max(dp[i-1],dp[i-2]+nums[i])
3	House Robber II	213	Medium	Circular: run twice
4	Coin Change	322	Medium	dp[i]=min coins for amount i
5	Coin Change II	518	Medium	Unbounded knapsack 1D
6	Longest Increasing Subsequence	300	Medium	dp[i]=LIS ending at i
7	Maximum Product Subarray	152	Medium	Track running max and min
8	Decode Ways	91	Medium	dp[i] with 1/2 digit check
9	Word Break	139	Medium	dp[i]=can segment s[0..i]
10	Perfect Squares	279	Medium	BFS / DP min squares
11	Integer Break	343	Medium	dp product split
12	Longest Palindromic Substring	5	Medium	Expand around center / dp
13	Palindromic Substrings	647	Medium	Expand around center count
14	Delete and Earn	740	Medium	House Robber transform
15	Min Cost Climbing Stairs	746	Easy	Simple 1D dp
16	Domino and Tromino Tiling	790	Medium	Tiling recurrence
17	Maximum Length of Repeated Subarray	718	Medium	1D rolling dp
18	Longest Arithmetic Subsequence	1027	Medium	dp with hashmap of diffs

Dynamic Programming - 2D

Tip: 0/1 Knapsack and LCS are the two templates behind most 2D DP interview questions.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Unique Paths	62	Medium	dp[i][j]=dp[i-1][j]+dp[i][j-1]
2	Unique Paths II	63	Medium	Grid dp with obstacles
3	Minimum Path Sum	64	Medium	2D DP grid traversal
4	Longest Common Subsequence	1143	Medium	Classic LCS dp[i][j]
5	Edit Distance	72	Medium	Classic Levenshtein DP
6	Target Sum	494	Medium	DP or DFS + memo
7	Partition Equal Subset Sum	416	Medium	0/1 Knapsack
8	Interleaving String	97	Medium	2D string DP
9	Distinct Subsequences	115	Hard	2D DP counting
10	Burst Balloons	312	Hard	Interval DP

#	Problem Name	LC No.	Difficulty	Key Concept
11	Maximal Square	221	Medium	2D dp on grid
12	Regular Expression Matching	10	Hard	2D DP with wildcard
13	Wildcard Matching	44	Hard	2D DP with wildcard
14	Minimum ASCII Delete Sum for Two Strings	712	Medium	2D dp cost variant
15	Longest Palindromic Subsequence	516	Medium	2D dp on reversed LCS
16	Cherry Pickup	741	Hard	3D dp, two travelers
17	Dungeon Game	174	Hard	Reverse 2D dp
18	Knapsack 0/1 (GfG classic)	N/A	Medium	Foundational knapsack template

Advanced DP (Bitmask / Interval / State Machine)

Tip: These show up in dream-tier and product-company rounds once you've nailed the basics.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Best Time to Buy and Sell Stock with Cooldown	309	Medium	State machine DP
2	Best Time to Buy and Sell Stock III	123	Hard	State machine DP, 2 transactions
3	Best Time to Buy and Sell Stock IV	188	Hard	State machine DP, k transactions
4	Partition to K Equal Sum Subsets	698	Medium	Bitmask DP / backtrack
5	Shortest Path Visiting All Nodes	847	Hard	Bitmask BFS
6	Matrix Chain Multiplication (GfG classic)	N/A	Hard	Interval DP template
7	Palindrome Partitioning II	132	Hard	Interval DP min cuts
8	Stone Game	877	Medium	Interval / game DP
9	Minimum Cost to Cut a Stick	1547	Hard	Interval DP
10	Profitable Schemes	879	Hard	Knapsack with 2 constraints

PHASE 7 Graphs - BFS + DFS + Union Find + Shortest Path

Week 21-24

Advanced pattern for product companies and dream-tier MNCs

Graph representation in Java: adjacency list via List<List>. Always clarify: directed/undirected? weighted? connected? These change the approach completely.

BFS / DFS on Graphs

Tip: Number of Islands (LC 200) is the most commonly asked graph problem in Indian MNC drives.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Number of Islands	200	Medium	DFS/BFS flood fill
2	Clone Graph	133	Medium	BFS + HashMap
3	Max Area of Island	695	Medium	DFS accumulate area
4	Rotting Oranges	994	Medium	Multi-source BFS
5	Course Schedule	207	Medium	Cycle detection DFS
6	Course Schedule II	210	Medium	Topological sort BFS
7	Pacific Atlantic Water Flow	417	Medium	Reverse BFS from borders
8	Surrounded Regions	130	Medium	DFS from edges
9	Shortest Path in Binary Matrix	1091	Medium	BFS shortest path
10	Walls and Gates	286	Medium	Multi-source BFS
11	Flood Fill	733	Easy	DFS/BFS grid fill
12	Word Ladder	127	Hard	BFS on word graph
13	Word Ladder II	126	Hard	BFS + backtrack path build
14	01 Matrix	542	Medium	Multi-source BFS
15	Graph Valid Tree	261	Medium	DFS/Union Find cycle check
16	Is Graph Bipartite	785	Medium	BFS/DFS 2-coloring
17	All Paths From Source to Target	797	Medium	DFS backtrack on DAG
18	Reconstruct Itinerary	332	Hard	Eulerian path DFS

Union Find (Disjoint Set)

Tip: find(x) with path compression + union(x,y) with rank makes both operations near O(1).

#	Problem Name	LC No.	Difficulty	Key Concept
1	Number of Provinces	547	Medium	Union Find components
2	Redundant Connection	684	Medium	Detect cycle via UF
3	Accounts Merge	721	Medium	UF on emails
4	Number of Operations to Make Network Connected	1319	Medium	UF components
5	Number of Connected Components in an Undirected Graph	323	Medium	UF or BFS
6	Satisfiability of Equality Equations	990	Medium	UF on constraints

#	Problem Name	LC No.	Difficulty	Key Concept
7	Most Stones Removed with Same Row or Column	947	Medium	UF row/col union
8	Smallest String With Swaps	1202	Medium	UF + grouping
9	Swim in Rising Water	778	Hard	UF / binary search hybrid
10	Regions Cut By Slashes	959	Medium	UF on subdivided grid

Shortest Path (Dijkstra / Bellman-Ford)

Tip: Dijkstra = BFS + min-heap for weighted graphs with non-negative edges.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Network Delay Time	743	Medium	Dijkstra classic
2	Path With Minimum Effort	1631	Medium	Dijkstra on grid
3	Swim in Rising Water	778	Hard	Dijkstra / binary search
4	Cheapest Flights Within K Stops	787	Medium	Bellman-Ford / BFS
5	Path with Maximum Probability	1514	Medium	Dijkstra max variant
6	Number of Ways to Arrive at Destination	1976	Medium	Dijkstra + path counting
7	Minimum Cost to Make at Least One Valid Path in a Grid	1368	Hard	0-1 BFS
8	Find the City With the Smallest Number of Neighbors	1334	Medium	Floyd-Warshall

Topological Sort & MST

Tip: Kahn's algorithm (BFS + in-degree) is usually easier to code under pressure than DFS-based topo sort.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Course Schedule II	210	Medium	Kahn's BFS topo sort
2	Alien Dictionary	269	Hard	Topo sort from chars
3	Minimum Height Trees	310	Medium	Leaf peeling = topo
4	Sequence Reconstruction	444	Medium	Unique topo sort check
5	Min Cost to Connect All Points	1584	Medium	MST - Prim's/Kruskal's
6	Connecting Cities With Minimum Cost	1135	Medium	MST - Kruskal's + UF
7	Parallel Courses	1136	Medium	Topo sort levels

PHASE 8 Bit Manipulation + Math + Design Patterns (DSA)

Week 25-26

Rounding out the pattern toolkit before mock interviews

Bit Manipulation

Tip: Know these cold: n & (n-1) removes the lowest set bit. n & 1 checks odd/even. XOR cancels pairs.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Single Number	136	Easy	XOR cancels pairs
2	Single Number II	137	Medium	Bit counting mod 3
3	Single Number III	260	Medium	XOR + partition by bit
4	Number of 1 Bits	191	Easy	n & (n-1) trick
5	Counting Bits	338	Easy	DP + bit trick
6	Reverse Bits	190	Easy	Bit shifting
7	Missing Number	268	Easy	XOR / Gauss formula
8	Sum of Two Integers	371	Medium	Bitwise add (no + operator)
9	Bitwise AND of Numbers Range	201	Medium	Common prefix shifting
10	Maximum XOR of Two Numbers in an Array	421	Medium	Binary trie
11	Power of Two	231	Easy	n & (n-1) == 0
12	Power of Four	342	Easy	Bitmask check
13	UTF-8 Validation	393	Medium	Bit pattern checking
14	Total Hamming Distance	477	Medium	Bit-position counting
15	Subsets	78	Medium	Bitmask enumeration

Math & Number Theory

Tip: GCD/LCM, sieve of Eratosthenes, and modular arithmetic show up more than people expect.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Reverse Integer	7	Medium	Digit reversal + overflow check
2	Palindrome Number	9	Easy	Digit reversal check
3	Count Primes	204	Medium	Sieve of Eratosthenes
4	Greatest Common Divisor of Strings	1071	Easy	GCD / Euclidean algorithm
5	Excel Sheet Column Number	171	Easy	Base-26 conversion
6	Pow(x, n)	50	Medium	Fast exponentiation
7	Sqrt(x)	69	Easy	Newton's method / BS
8	Happy Number	202	Easy	Cycle detection with math
9	Factorial Trailing Zeroes	172	Medium	Count factors of 5
10	Rectangle Area	223	Medium	Geometry + overlap math
11	Basic Calculator II	227	Medium	Expression parsing math
12	Random Pick with Weight	528	Medium	Prefix sum + binary search

Design (Data Structure Design)

Tip: These test whether you can combine 2+ patterns cleanly under a clean API — common in Tier-2/3 rounds.

#	Problem Name	LC No.	Difficulty	Key Concept
1	LRU Cache	146	Medium	HashMap + Doubly Linked List
2	LFU Cache	460	Hard	HashMap + frequency buckets
3	Design Twitter	355	Medium	HashMap + heap merge
4	Insert Delete GetRandom O(1)	380	Medium	HashMap + ArrayList swap-delete
5	Design Underground System	1396	Medium	HashMap based tracking
6	Time Based Key-Value Store	981	Medium	HashMap + binary search
7	Design a Leaderboard	1244	Medium	HashMap + sorting/heap
8	All O`one Data Structure	432	Hard	HashMap + doubly linked list
9	Design Snake Game	353	Medium	Queue + HashSet simulation
10	Design Search Autocomplete System	642	Hard	Trie + heap design

PHASE 9 SQL Query Patterns (LeetCode Database)

Parallel track - Week 10 onward

Every Java Full Stack interview includes at least one SQL round — practice these alongside DSA

Setup: practice on LeetCode's Database track (MySQL) in parallel with DP/Graphs. 30 min/day is enough.

Joins & Filtering

Tip: Master INNER, LEFT, and self-joins first — they cover 60% of SQL interview questions.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Combine Two Tables	175	Easy	LEFT JOIN
2	Employees Earning More Than Their Managers	181	Easy	Self JOIN
3	Duplicate Emails	182	Easy	GROUP BY + HAVING
4	Customers Who Never Order	183	Easy	LEFT JOIN + IS NULL
5	Rising Temperature	197	Easy	Self JOIN + DATEDIFF
6	Trips and Users	262	Hard	Multi-table JOIN + filtering
7	Department Highest Salary	184	Medium	JOIN + subquery MAX
8	Department Top Three Salaries	185	Hard	JOIN + window function

Aggregation & Window Functions

Tip: RANK(), DENSE_RANK(), and ROW_NUMBER() OVER(PARTITION BY...) are the most-asked window functions.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Second Highest Salary	176	Medium	Subquery / LIMIT OFFSET
2	Nth Highest Salary	177	Medium	DENSE_RANK() window fn
3	Rank Scores	178	Medium	DENSE_RANK() window fn
4	Consecutive Numbers	180	Medium	LAG/LEAD or self-join
5	Group Sold Products By The Date	1484	Easy	GROUP_CONCAT + GROUP BY
6	Exchange Seats	626	Medium	CASE WHEN + row parity
7	Median Employee Salary	569	Hard	Window function ranking
8	Cumulative Salary of an Employee	579	Hard	Window function sum

Subqueries & Set Operations

Tip: UNION removes duplicates; UNION ALL doesn't — know when each is correct.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Big Countries	595	Easy	WHERE with OR condition
2	Classes More Than 5 Students	596	Easy	GROUP BY + HAVING
3	Human Traffic of Stadium	601	Hard	Self JOIN + consecutive rows
4	Sales Person	607	Easy	NOT IN subquery
5	Tree Node	608	Medium	CASE WHEN + self join

#	Problem Name	LC No.	Difficulty	Key Concept
6	Actors and Directors Who Cooperated	1050	Easy	GROUP BY + HAVING count
7	Investments in 2016	585	Medium	Subquery + aggregate filtering

PHASE 10 Spring Boot + Backend Design Patterns

Parallel track - Week 12 onward

These are what interviewers probe once DSA rounds are cleared — build a small demo app per pattern

Rule: don't just read about these — implement each in a mini Spring Boot project (even a 3-endpoint CRUD app counts).

Core Spring Concepts

Tip: Be ready to explain the 'why', not just the annotation, for each of these in an interview.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Dependency Injection (constructor vs field)	-	Core	IoC container, @Autowired
2	Bean Lifecycle	-	Core	@PostConstruct, @PreDestroy
3	Spring Bean Scopes	-	Core	singleton vs prototype vs request
4	Layered Architecture	-	Core	Controller -> Service -> Repository
5	DTO vs Entity Separation	-	Core	Avoid leaking JPA entities via API
6	Exception Handling	-	Core	@ControllerAdvice, @ExceptionHandler
7	Validation	-	Core	@Valid, Bean Validation (JSR-380)
8	Profiles & Config	-	Core	application-dev.yml vs application-prod.yml
9	Spring Security Basics	-	Core	JWT auth, filters, roles
10	Transactions	-	Core	@Transactional, rollback rules

Design Patterns to Implement

Tip: Recruiters at Zoho/Freshworks/dream-tier companies love asking 'show me a pattern you've used'.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Singleton Pattern	-	GoF	Spring beans are singleton by default
2	Factory Pattern	-	GoF	Object creation abstraction
3	Builder Pattern	-	GoF	Complex object construction (DTOs)
4	Repository Pattern	-	Architectural	Spring Data JPA repositories
5	Strategy Pattern	-	GoF	Interchangeable algorithms via interface
6	Observer Pattern	-	GoF	Spring ApplicationEvent / Listener
7	Decorator Pattern	-	GoF	Wrapping behavior, filters/interceptors
8	Proxy Pattern	-	GoF	Spring AOP proxies, @Transactional internals
9	Adapter Pattern	-	GoF	Wrapping third-party APIs cleanly
10	Template Method Pattern	-	GoF	JdbcTemplate, RestTemplate style

REST API & Persistence

Tip: Practice building the same CRUD resource 3 times: with JPA, with JDBC, and with a caching layer.

#	Problem Name	LC No.	Difficulty	Key Concept
1	RESTful CRUD API design	-	Practice	Proper verbs, status codes, versioning
2	Pagination & Sorting	-	Practice	Pageable, Sort in Spring Data
3	Exception -> Proper HTTP status mapping	-	Practice	404 vs 400 vs 409 vs 500
4	JPA Relationships	-	Practice	@OneToMany, @ManyToOne, @ManyToMany
5	N+1 Query Problem	-	Practice	Fetch joins, @EntityGraph
6	Caching with @Cacheable	-	Practice	Reduce DB load on hot reads
7	File Upload/Download API	-	Practice	MultipartFile handling
8	API Rate Limiting (basic)	-	Practice	Bucket4j or custom filter
9	Idempotent PUT vs POST	-	Practice	REST semantics under load
10	Testing with JUnit + Mockito	-	Practice	Unit test service layer with mocks

PHASE 11 System Design Basics (For Dream-Tier Rounds)

Final 4-6 weeks before placement season

Walmart/SAP Labs/Razorpay-style interviews test this after DSA clears

You don't need to be an expert — you need to explain trade-offs clearly for a junior-level system.

Fundamentals

Tip: Practice explaining each of these in 2 minutes, out loud, like you would in an interview.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Vertical vs Horizontal Scaling	-	Concept	When to scale up vs out
2	Load Balancing	-	Concept	Round robin, least connections
3	Caching Strategies	-	Concept	Cache-aside, write-through, TTL
4	Database Indexing	-	Concept	B-Tree index, query speedup trade-offs
5	SQL vs NoSQL	-	Concept	When to choose which, CAP theorem basics
6	Database Sharding	-	Concept	Horizontal partitioning strategies
7	Message Queues	-	Concept	Kafka/RabbitMQ, async decoupling
8	Rate Limiting Algorithms	-	Concept	Token bucket, sliding window
9	CDN Basics	-	Concept	Static asset delivery, edge caching
10	Consistent Hashing	-	Concept	Distributed cache/load balancing

Mini Design Practice

Tip: For each, sketch: requirements -> high-level design -> DB schema -> API contract -> bottlenecks.

#	Problem Name	LC No.	Difficulty	Key Concept
1	Design a URL Shortener	-	Practice	Hashing, DB schema, redirect flow
2	Design a Rate Limiter	-	Practice	Token bucket implementation
3	Design a Parking Lot System	-	Practice	OOP + concurrency basics
4	Design LIBRO-BOT's Backend at Scale	-	Practice	Your own project — great interview story
5	Design a Notification System	-	Practice	Queue + fan-out + retries
6	Design an E-commerce Cart Service	-	Practice	Consistency vs availability trade-off
7	Design a Chat Application (basic)	-	Practice	WebSockets, message delivery
8	Design a Job Scheduler	-	Practice	Cron-like design, retries, idempotency

GITHUB REPO SETUP GUIDE

Repo names: **leetcode-solutions** (DSA), **campus-placement-prep** (notes, SQL, Spring Boot mini-projects, resume, mock interview logs).

Folder / File	Purpose
README.md	Progress table, LeetCode stats card, 'Currently solving' badge
/arrays/, /two-pointers/, /sliding-window/	Phase 1-2 problems
/hashing/, /stack-queue/	Phase 3 problems
/recursion-backtrack/, /linked-list/	Phase 4 problems
/trees/, /trie/	Phase 5 problems
/heap/, /greedy/, /dp/	Phase 6 problems
/graphs/	Phase 7 problems
/bit-manipulation/, /math/, /design/	Phase 8 problems
/sql/	Phase 9 queries with schema + explanation
/spring-boot-demos/	One small demo repo per backend pattern (Phase 10)
/system-design-notes/	Markdown notes + diagrams per topic (Phase 11)
notes.md (in each folder)	Your own notes: pattern, approach, time complexity

Daily Git workflow:

```
git add .
git commit -m 'LC 200 - Number of Islands - DFS solution'
git push origin main
```

Commit message format: 'LC [number] - [Problem Name] - [approach]'. For Spring Boot/SQL commits use: 'SPRING - [pattern] - [what you built]' or 'SQL - [problem] - [approach]'.

TOP RESOURCES + STUDY SCHEDULE

Resource	Link	Use it for
NeetCode.io	neetcode.io	Best free pattern-based roadmap, organized like this PDF.
NeetCode YouTube	youtube.com/@NeetCode	Video explanations. Watch AFTER you attempt.
Striver's A2Z Sheet	takeuforward.org	Indian placement-focused DSA sheet, 455 problems.
LeetCode	leetcode.com	Primary practice platform + SQL/Database track.
GeeksforGeeks	geeksforgeeks.org	Theory + Java code for every DSA concept.
Grokking the Coding Interview	educative.io	Pattern-based guide, 16 patterns.
Java Docs	docs.oracle.com	Official Java 17 docs. Bookmark HashMap, ArrayList, PriorityQueue.
Spring.io Guides	spring.io/guides	Official short guides for every Spring Boot pattern.
Baeldung	baeldung.com	Best practical Spring Boot + Java tutorials for interviews.
System Design Primer	github.com/donnemartin/system-design-primer	Free, thorough system design fundamentals.

Time Block	Activity
Day 1-2 of each pattern	Watch a NeetCode/Baeldung explainer for the pattern (theory only)
Day 3 onward	Attempt 2 Easy/Medium problems — no hints for first 20 min
After solving	Write your approach in notes.md. Commit to GitHub.
Weekend	Revise 5 problems from the previous week. Re-solve without looking.
Every 2 weeks	Mock interview: 2 random DSA problems, 45-min timer, no hints.
Every 3-4 weeks	Build/extend one Spring Boot mini-feature using a pattern from Phase 10.

COMPANY-WISE TARGETING STRATEGY

Tier 1 — Mass Recruiters

Focus: Phases 1-4 + Aptitude

Companies: TCS · Infosys · Wipro · Cognizant · Capgemini

Array, String, Basic DP, LinkedList. Mostly Easy-Medium. Aptitude matters equally. TCS uses TCS NQT; Infosys uses HackerRank (2 problems, 90 min). Get these first — your safety net while you upskill.

Tier 2 — Mid-tier Product

Focus: Phases 1-6 + SQL + Spring Boot basics

Companies: Zoho · Freshworks · Hexaware · MindTree · L&T; Infotech

Zoho is famous for tough DSA rounds (Stack, DP, Trees) across 5-6 rounds including a full-day coding round. Freshworks focuses on Spring Boot + DSA combo, Medium-Hard problems. LIBRO-BOT + Spring Boot roadmap is your edge.

Tier 3 — Dream Companies

Focus: All 11 phases + System Design

Companies: Walmart Global Tech · SAP Labs · ThoughtWorks · Razorpay

Requires Phases 1-11 complete + strong projects + GitHub activity. Walmart: DSA (hard), LLD, system design. SAP Labs: Java internals + OOP + DSA. ThoughtWorks: clean code + pair programming + TDD mindset.

PLACEMENT TIPS: LOW CGPA GAME PLAN

Low marks + 10 LPA = absolutely possible, but ONLY if your profile compensates with visible, verifiable proof of skill. Here is the exact strategy:

YOUR ACE CARD	**LIBRO-BOT — Best Startup at SREC INNOVATE 2026** Python + OpenCV + React + Firebase. Lead with this in every interview. Practice explaining it in 2 minutes: what it does, why you built it, the tech stack, and what you learned. This beats an 8.5 CGPA with no projects.
MAKE IT VISIBLE	**GitHub green squares + LinkedIn activity** Recruiters Google your name. 180 days of green squares tells them you code every day — rarer than high marks. Post 1 LinkedIn update per week under #LearningInPublic and #JavaDeveloper.
CERTIFICATIONS	**ServiceNow University AI cert + Accenture Digital Skills AI (96%)** Put these under Certifications. Add more: AWS Cloud Practitioner (free voucher via AWS Educate), Oracle Java SE Foundations (free), Meta Frontend Developer cert.
INTERNSHIP PROOF	**Your current paid Java internship at the coaching institute** Even if small, it's real experience. On resume: 'Java Programming Intern | [Institute Name] | [Month Year – Month Year]'. Describe: 'Built OOP-based applications using Core Java, implemented CRUD with MySQL and JDBC.'
BACKEND DEPTH	**Spring Boot mini-projects from Phase 10** Two or three small but complete Spring Boot demos (auth, CRUD + caching, a design-pattern showcase) prove you can ship, not just solve LeetCode — this is what separates you at Tier 2/3 interviews.
APPLY SMART	**Referrals > Direct applications** Connect with SREC pass-outs now in TCS/Infosys on LinkedIn. Message them for guidance. Referrals bypass resume screening — this is how low-CGPA candidates get interviews.

Final message: Your batch mates with 8+ CGPA and zero projects will apply to 100 companies and get filtered in the first round. You — with LIBRO-BOT, daily GitHub, DSA, SQL, and Spring Boot patterns under your belt — will pass the coding rounds and walk into interviews with a story to tell. Play the long game.